● COMPLETE · PASS

Assessment Service

assess-svc

Exam Lifecycle & Auto-Grading

STATUS	TESTS	BUILD	GENERATED
Frozen	PASS Passing	SUCCESS	March 07, 2026

assess-svc — COMPLETION DOCUMENT

STATUS: FROZEN — IMMUTABLE AFTER 2026-03-07
This document is a permanent, authoritative record of the assess-svc microservice as delivered and verified. It must never be edited after creation.

0. Vital Statistics

ATTRIBUTE	VALUE
Service name	assess-svc
Maven artifact	`com.edutech:assess-svc:1.0.0-SNAPSHOT`
Root package	`com.edutech.assess`
DB schema	`assess_schema`
Flyway migrations	V1–V7 (7 scripts)
Source files written	92 (Domain+App: ~56, Infra: ~19, API+Bootstrap+Tests: ~17)
Test results	**11/11 PASS** — BUILD SUCCESS in 6.412 s
ArchUnit rules	5/5 PASS
Service tests	6/6 PASS (SubmissionServiceTest)
Build method	3 parallel specialist agents
Spring Boot version	3.x (Jakarta EE 10, Hibernate 6.x, virtual threads)
Java version	17
Architecture pattern	Hexagonal (Ports & Adapters), strict layer isolation

1. Bounded Context

assess-svc owns the **Assessment & Grading** bounded context within the EduTech AI Platform. It is the single source of truth for:

- Exam lifecycle (DRAFT → PUBLISHED → CLOSED / CANCELLED)
- Question bank with IRT parameters (difficulty `b`, discrimination `a`, guessing `c`) and vector embedding storage
- Student enrollment per exam
- Submission tracking with attempt-number enforcement
- Atomic auto-grading: per-question correctness, scored marks accumulation, percentage calculation, letter-grade assignment
- Grade records — one per submission, immutable after creation
- Real-time exam session infrastructure (WebSocket/STOMP — Phase 1 stub)
- Computer Adaptive Testing (CAT) engine configuration (Phase 2)

assess-svc does **not** own student identity, center/batch metadata, or fee data. It references those entities by UUID only, never joining cross-service tables.

---

2. Architecture Philosophy

Hexagonal Architecture — Enforced by ArchUnit

```
┌──────────────────────────────────────────────────────────┐
│  api (REST controllers, global exception handler)         │
├──────────────────────────────────────────────────────────┤
│  application (services, DTOs, exceptions, config ports)   │
├──────────────────────────────────────────────────────────┤
│  domain (entities, enums, events, ports IN/OUT)           │
├──────────────────────────────────────────────────────────┤
│  infrastructure (persistence, security, messaging, WS)    │
└──────────────────────────────────────────────────────────┘
```

Dependency rule (immutable):
- `domain` depends on NOTHING (zero Spring, zero Jakarta, except JPA annotations)
- `application` depends on `domain` only
- `infrastructure` depends on `domain` and `application`
- `api` depends on `application` and `domain`
- `infrastructure` does NOT depend on `api`
- `api` does NOT depend on `infrastructure`

Five ArchUnit Rules (all PASS)

RULE	DESCRIPTION
1	`domain` classes must not depend on `infrastructure` or `api`
2	`application` classes must not depend on `infrastructure` or `api`
3	`infrastructure` classes must not depend on `api`
4	`api` classes must not depend on `infrastructure`
5	Services must reside in `application.service` package

Design Invariants

INVARIANT	IMPLEMENTATION
No Lombok	Manual constructors, no annotations

INVARIANT	IMPLEMENTATION
No hardcoded values	All config via `${ENV_VAR}` in `application.yml`
Soft delete	`deleted_at IS NULL` on all mutable-entity JPQL queries
Optimistic locking	`@Version Long version` on all mutable entities
Static factory pattern	`Entity.create(...)` sets `id = UUID.randomUUID()` pre-persist
UUID generation	`@GeneratedValue(strategy = GenerationType.UUID)` — Hibernate 6.x uses pre-set value
Best-effort Kafka	Publish failures logged, never roll back DB transaction
RFC 7807 errors	`ProblemDetail.forStatusAndDetail(status, detail)` with `https://edutech.com/problems/{type}`
Immutable SubmissionAnswer	`all columns updatable=false`, no `@Version`, no `updatedAt`, no `deletedAt`
Constructor injection	All Spring beans use constructor injection exclusively
Virtual threads	`spring.threads.virtual.enabled: true`

3. Complete Package Structure (92 files)

```
com.edutech.assess
├── AssessSvcApplication.java                    @SpringBootApplication @ConfigurationPropertiesScan
│
├── domain/
│   ├── model/
│   │   ├── Exam.java                            @Entity — lifecycle owner
│   │   ├── Question.java                        @Entity — IRT params + embedding
│   │   ├── ExamEnrollment.java                  @Entity — student↔exam link
│   │   ├── Submission.java                      @Entity — attempt tracking
│   │   ├── SubmissionAnswer.java                @Entity — IMMUTABLE answer record
│   │   ├── Grade.java                           @Entity — one per submission
│   │   ├── ExamMode.java                        STANDARD | CAT
│   │   ├── ExamStatus.java                      DRAFT | PUBLISHED | CLOSED | CANCELLED
│   │   ├── EnrollmentStatus.java                ENROLLED | WITHDRAWN
│   │   ├── SubmissionStatus.java                IN_PROGRESS | GRADED | INVALIDATED
│   │   ├── QuestionType.java                    MCQ | TRUE_FALSE | SHORT_ANSWER
│   │   └── Role.java                            STUDENT | CENTER_ADMIN | SUPER_ADMIN
│   ├── event/
│   │   ├── ExamPublishedEvent.java              record
│   │   ├── ExamSubmittedEvent.java              record
│   │   └── GradeIssuedEvent.java                record
│   └── port/
│       ├── in/
│       │   ├── CreateExamUseCase.java
│       │   ├── PublishExamUseCase.java
│       │   ├── AddQuestionUseCase.java
│       │   ├── EnrollStudentUseCase.java
│       │   ├── StartSubmissionUseCase.java
│       │   └── SubmitAnswersUseCase.java
│       └── out/
│           ├── ExamRepository.java
│           ├── QuestionRepository.java
│           ├── ExamEnrollmentRepository.java
│           ├── SubmissionRepository.java
│           ├── SubmissionAnswerRepository.java
```

```
│           ├── GradeRepository.java
│           └── AssessEventPublisher.java
│
├── application/
│   ├── config/
│   │   ├── JwtProperties.java                      @ConfigurationProperties("jwt")
│   │   └── CatProperties.java                      @ConfigurationProperties("cat")
│   ├── dto/
│   │   ├── AuthPrincipal.java                      record — userId,email,role,centerId,deviceFP
│   │   ├── CreateExamRequest.java                  record + Bean Validation
│   │   ├── ExamResponse.java                       record
│   │   ├── AddQuestionRequest.java                 record + Bean Validation
│   │   ├── QuestionResponse.java                   record
│   │   ├── EnrollStudentRequest.java               record
│   │   ├── EnrollmentResponse.java                 record
│   │   ├── AnswerEntry.java                        record — questionId, selectedOption
│   │   ├── SubmitAnswersRequest.java               record — List<AnswerEntry>
│   │   ├── SubmissionResponse.java                 record
│   │   └── GradeResponse.java                      record
│   ├── exception/
│   │   ├── AssessException.java                    abstract base (RuntimeException)
│   │   ├── ExamNotFoundException.java
│   │   ├── QuestionNotFoundException.java
│   │   ├── EnrollmentNotFoundException.java
│   │   ├── SubmissionNotFoundException.java
│   │   ├── AssessAccessDeniedException.java
│   │   ├── ExamNotPublishedException.java
│   │   ├── DuplicateEnrollmentException.java
│   │   ├── SubmissionAlreadySubmittedException.java
│   │   └── MaxAttemptsExceededException.java
│   └── service/
│       ├── ExamService.java                        implements CreateExamUseCase, PublishExamUseCase
│       ├── QuestionService.java                    implements AddQuestionUseCase (uses ObjectMapper)
│       ├── EnrollmentService.java                  implements EnrollStudentUseCase
│       ├── SubmissionService.java                  implements StartSubmissionUseCase, SubmitAnswersUseCase
│       └── GradeService.java                       read-only grade queries
│
├── infrastructure/
│   ├── config/
│   │   └── KafkaTopicProperties.java               @ConfigurationProperties("kafka.topics")
│   ├── security/
│   │   ├── JwtTokenValidator.java                  JJWT 0.12.x RS256
│   │   ├── JwtAuthenticationFilter.java            OncePerRequestFilter + MDC
│   │   └── SecurityConfig.java                     stateless, @EnableMethodSecurity
│   ├── websocket/
│   │   └── WebSocketConfig.java                    @EnableWebSocketMessageBroker, STOMP stub
│   ├── persistence/
│   │   ├── SpringDataExamRepository.java           package-private JpaRepository
│   │   ├── ExamPersistenceAdapter.java             @Component implements ExamRepository
│   │   ├── SpringDataQuestionRepository.java       package-private
│   │   ├── QuestionPersistenceAdapter.java         @Component
│   │   ├── SpringDataExamEnrollmentRepository.java package-private
│   │   ├── ExamEnrollmentPersistenceAdapter.java   @Component
│   │   ├── SpringDataSubmissionRepository.java     package-private
│   │   ├── SubmissionPersistenceAdapter.java       @Component
│   │   ├── SpringDataSubmissionAnswerRepository.java package-private
│   │   ├── SubmissionAnswerPersistenceAdapter.java @Component
│   │   ├── SpringDataGradeRepository.java          package-private
│   │   └── GradePersistenceAdapter.java            @Component
│   └── messaging/
```

```
│       ├── AssessEventKafkaAdapter.java          implements AssessEventPublisher
│       └── CenterEventConsumer.java              @KafkaListener(center-events)
│
├── api/
│   ├── GlobalExceptionHandler.java               @RestControllerAdvice RFC 7807
│   ├── ExamController.java                       /api/v1/exams
│   ├── QuestionController.java                   /api/v1/exams/{examId}/questions
│   ├── EnrollmentController.java                 /api/v1/exams/{examId}/enrollments
│   ├── SubmissionController.java                 /api/v1/exams/{examId}/submissions
│   └── GradeController.java                      /api/v1/grades
│
└── [test]
    ├── architecture/
    │   └── ArchitectureRulesTest.java            5 ArchUnit rules
    └── application/service/
        └── SubmissionServiceTest.java            6 Mockito unit tests
```

---

4. Domain Model — Deep Specification

4.1 Exam Entity

```
Exam
├── id:           UUID          (PK, immutable)
├── batchId:      UUID          (immutable — cross-service ref)
├── centerId:     UUID          (immutable — cross-service ref)
├── title:        String
├── description:  String        (TEXT)
├── mode:         ExamMode      (STANDARD | CAT)
├── durationMinutes: int
├── maxAttempts:  int           (>0, DB constraint)
├── startAt:      Instant       (nullable — open-ended until set)
├── endAt:        Instant       (nullable)
├── totalMarks:   double        (>0, DB constraint)
├── passingMarks: double        (0 ≤ passingMarks ≤ totalMarks, DB constraint)
├── status:       ExamStatus    (DRAFT initial)
├── version:      Long          (@Version)
├── createdAt:    Instant       (immutable)
├── updatedAt:    Instant
└── deletedAt:    Instant       (soft delete)
```

State machine:

```
DRAFT ──publish()──→ PUBLISHED ──close()──→ CLOSED
  │                     │
  └──cancel()──→ CANCELLED  ←──cancel()──┘
                                (CLOSED cannot be cancelled — guards enforced in entity)
```

Factory: `Exam.create(batchId, centerId, title, description, mode, durationMinutes, maxAttempts, startAt, endAt, totalMarks, passingMarks)` → status=DRAFT

4.2 Question Entity

```
Question
├── id:           UUID     (PK, immutable)
```

```
├── examId:         UUID     (immutable)
├── questionText:   String   (TEXT)
├── optionsJson:    String   (TEXT — serialized List<String>, handled in QuestionService)
├── correctAnswer:  int      (0-based index into options)
├── explanation:    String   (TEXT, nullable)
├── marks:          double
├── difficulty:     double   (IRT parameter b — difficulty)
├── discrimination: double   (IRT parameter a — discrimination)
├── guessingParam:  double   (IRT parameter c — guessing)
├── embeddingJson:  String   (TEXT — placeholder for pgvector, null until AI enrichment)
├── version:        Long
├── createdAt:      Instant
├── updatedAt:      Instant
└── deletedAt:      Instant
```

IRT parameters explanation:

- `difficulty (b)`: The point on the ability scale where a student has 50% probability of answering correctly
- `discrimination (a)`: How well the item differentiates between high/low ability students
- `guessingParam (c)`: Lower asymptote — probability of correct answer by pure guessing

Options serialization: `QuestionService` uses Jackson `ObjectMapper` to serialize `List<String>` → `optionsJson` on write, and deserialize on read. This keeps the `domain.model.Question` free of Jackson annotations while maintaining full option text in a structured format. ObjectMapper is a non-Spring dependency; it does not violate ArchUnit rules.

4.3 ExamEnrollment Entity

```
ExamEnrollment
├── id:         UUID             (PK)
├── examId:     UUID             (immutable)
├── studentId:  UUID             (immutable)
├── status:     EnrollmentStatus (ENROLLED | WITHDRAWN)
├── enrolledAt: Instant          (immutable)
├── version:    Long
├── createdAt:  Instant
└── updatedAt:  Instant
```

State: `ENROLLED → withdraw() → WITHDRAWN`

4.4 Submission Entity

```
Submission
├── id:            UUID             (PK)
├── examId:        UUID             (immutable)
├── studentId:     UUID             (immutable)
├── enrollmentId:  UUID             (immutable)
├── attemptNumber: int              (immutable — set at creation)
├── startedAt:     Instant          (immutable)
├── submittedAt:   Instant          (set by grade())
├── totalMarks:    double           (set by grade())
├── scoredMarks:   double           (set by grade())
├── percentage:    double           (computed by grade())
├── status:        SubmissionStatus (IN_PROGRESS → GRADED | INVALIDATED)
├── version:       Long
├── createdAt:     Instant
└── updatedAt:     Instant
```

State machine:

```
IN_PROGRESS ──grade(scoredMarks, totalMarks)──→ GRADED
IN_PROGRESS ──invalidate()──────────────────→ INVALIDATED
```

grade() computation: `percentage = (scoredMarks / totalMarks) × 100.0`

4.5 SubmissionAnswer Entity — IMMUTABLE

```
SubmissionAnswer
├── id:             UUID     (PK)
├── submissionId:   UUID     (immutable, FK → submissions)
├── questionId:     UUID     (immutable, FK → questions)
├── selectedOption: int      (immutable — student's chosen index)
├── isCorrect:      boolean  (immutable — computed at submission time)
├── marksAwarded:   double   (immutable — 0.0 if wrong, q.marks if correct)
└── answeredAt:     Instant  (immutable)
```

Invariants enforced at entity and DB level:

- All `@Column` annotations have `updatable = false`
- No `@Version` (no version column)
- No `updatedAt` field
- No `deletedAt` field
- DB: `NO updated_at` column, BRIN index on `created_at` for append-only efficiency
- DB: `UNIQUE(submission_id, question_id)` — one answer per question per submission

Factory: `SubmissionAnswer.mark(submissionId, questionId, selectedOption, isCorrect, marksAwarded)`

4.6 Grade Entity

```
Grade
├── id:           UUID     (PK)
├── submissionId: UUID     (UNIQUE — one grade per submission, immutable)
├── studentId:    UUID     (immutable)
├── examId:       UUID     (immutable)
├── batchId:      UUID     (immutable)
├── centerId:     UUID     (immutable)
├── percentage:   double
├── letterGrade:  String   ("A"|"B"|"C"|"D"|"F", DB check constraint)
├── passed:       boolean
├── version:      Long
├── createdAt:    Instant
└── updatedAt:    Instant
```

Letter grade computation (in Grade.create):

```
percentage ≥ 90 → "A"
percentage ≥ 80 → "B"
percentage ≥ 70 → "C"
percentage ≥ 60 → "D"
else            → "F"
```

Passed determination: `percentage >= passingPercentage` where `passingPercentage = (exam.passingMarks / exam.totalMarks) × 100.0`

5. Domain Events

All events are Java `record` types. Published best-effort — Kafka failures logged, never rolled back.

EVENT	PUBLISHER	PAYLOAD FIELDS
`ExamPublishedEvent`	ExamService.publishExam()	examId, batchId, centerId, title, totalMarks
`ExamSubmittedEvent`	SubmissionService.submitAnswers()	submissionId, examId, studentId, scoredMarks, totalMarks
`GradeIssuedEvent`	SubmissionService.submitAnswers()	gradeId, submissionId, examId, studentId, batchId, centerId, percentage, passed

Note: `submitAnswers()` publishes **two events atomically within one** `@Transactional` **call:** `ExamSubmittedEvent` first, then `GradeIssuedEvent`. Both are emitted after DB commit succeeds.

---

6. Ports Specification

6.1 Ports IN (Use Cases)

INTERFACE	METHOD	IMPLEMENTOR
`CreateExamUseCase`	`createExam(CreateExamRequest, AuthPrincipal): ExamResponse`	ExamService
`PublishExamUseCase`	`publishExam(UUID examId, AuthPrincipal): ExamResponse`	ExamService
`AddQuestionUseCase`	`addQuestion(UUID examId, AddQuestionRequest, AuthPrincipal): QuestionResponse`	QuestionService
`EnrollStudentUseCase`	`enrollStudent(UUID examId, EnrollStudentRequest, AuthPrincipal): EnrollmentResponse`	EnrollmentService
`StartSubmissionUseCase`	`startSubmission(UUID examId, AuthPrincipal): SubmissionResponse`	SubmissionService
`SubmitAnswersUseCase`	`submitAnswers(UUID examId, UUID submissionId, SubmitAnswersRequest, AuthPrincipal): SubmissionResponse`	SubmissionService

6.2 Ports OUT (Repository + Publisher)

INTERFACE	KEY METHODS
`ExamRepository`	`findById`, `findByBatchId`, `save`
`QuestionRepository`	`findById`, `findByExamId`, `save`
`ExamEnrollmentRepository`	`findById`, `findByExamIdAndStudentId`, `save`
`SubmissionRepository`	`findById`, `countByExamIdAndStudentId`, `save`
`SubmissionAnswerRepository`	`saveAll(List<SubmissionAnswer>)`
`GradeRepository`	`findBySubmissionId`, `findByStudentId`, `save`
`AssessEventPublisher`	`publish(Object event)`

---

7. Application Services — Execution Traces

7.1 ExamService.createExam

```
Input: CreateExamRequest, AuthPrincipal (CENTER_ADMIN or SUPER_ADMIN)
1. Validate caller has access to the centerId — CENTER_ADMIN must have matching centerId
```

```
2. Exam.create(batchId, centerId, title, description, mode, durationMinutes,
               maxAttempts, startAt, endAt, totalMarks, passingMarks)
3. examRepository.save(exam)
4. Return ExamResponse
```

7.2 ExamService.publishExam

```
Input: examId, AuthPrincipal
1. examRepository.findById(examId) → or ExamNotFoundException
2. Verify caller is CENTER_ADMIN for exam.centerId, or SUPER_ADMIN
3. exam.publish() — guard: must be DRAFT, else IllegalStateException
4. examRepository.save(exam)
5. eventPublisher.publish(ExamPublishedEvent)
6. Return ExamResponse
```

7.3 SubmissionService.startSubmission — Full Trace

```
Input: examId, AuthPrincipal (STUDENT)
1. examRepository.findById(examId) → or ExamNotFoundException
2. Guard: exam.status == PUBLISHED → or ExamNotPublishedException
3. enrollmentRepository.findByExamIdAndStudentId(examId, studentId) → or EnrollmentNotFoundException
4. submissionRepository.countByExamIdAndStudentId(examId, studentId)
5. Guard: count < exam.maxAttempts → or MaxAttemptsExceededException
6. Submission.create(examId, studentId, enrollment.getId(), count + 1)
7. submissionRepository.save(submission)
8. Return SubmissionResponse(status=IN_PROGRESS, attemptNumber=count+1)
```

7.4 SubmissionService.submitAnswers — Full Trace (the heart of the service)

```
Input: examId, submissionId, SubmitAnswersRequest(List<AnswerEntry>), AuthPrincipal

@Transactional — ALL of this is one DB transaction:

1.  submissionRepository.findById(submissionId) → or SubmissionNotFoundException
2.  Guard: sub.studentId == principal.userId || principal.isSuperAdmin() → or AssessAccessDeniedException
3.  Guard: sub.status == IN_PROGRESS → or SubmissionAlreadySubmittedException
4.  examRepository.findById(examId) → or ExamNotFoundException
5.  For each AnswerEntry(questionId, selectedOption):
    a. questionRepository.findById(questionId) → or QuestionNotFoundException
    b. isCorrect = (selectedOption == question.correctAnswer)
    c. marksAwarded = isCorrect ? question.marks : 0.0
    d. SubmissionAnswer.mark(submissionId, questionId, selectedOption, isCorrect, marksAwarded)
6.  answerRepository.saveAll(submissionAnswers)              ← immutable insert only
7.  scoredMarks = SUM(answer.marksAwarded)
8.  sub.grade(scoredMarks, exam.totalMarks)
    └ percentage = scoredMarks / totalMarks × 100.0
    └ status → GRADED, submittedAt = now()
9.  submissionRepository.save(sub)
10. passingPct = (exam.passingMarks / exam.totalMarks) × 100.0
11. Grade.create(sub.id, sub.studentId, examId, exam.batchId, exam.centerId,
                 sub.percentage, passingPct)
    └ letterGrade computed inline (A/B/C/D/F)
    └ passed = percentage >= passingPct
12. gradeRepository.save(grade)

Post-commit (best-effort):
```

```
13. eventPublisher.publish(ExamSubmittedEvent)
14. eventPublisher.publish(GradeIssuedEvent)

15. log.info("Submission graded: id={} student={} score={}/{}")
16. Return SubmissionResponse(status=GRADED, scoredMarks, percentage, ...)
```

8. Authorization Matrix

ACTION	STUDENT	CENTER_ADMIN	SUPER_ADMIN
Create exam	✕	Own center only	✓ Any center
Publish exam	✕	Own center only	✓
Add question	✕	Own center only	✓
Enroll self	✓	✕	✓
Start submission	✓ Own only	✕	✓
Submit answers	✓ Own submission	✕	✓
View submission	✓ Own only	✕	✓
View grade	✓ Own only	✕	✓
View batch grades	✕	Own center	✓

AuthPrincipal record fields: `userId`, `email`, `role`, `centerId` (nullable), `deviceFP`
Helper methods: `belongsToCenter(UUID centerId)`, `isSuperAdmin()`, `userId()`

9. API Contract

ExamController — `/api/v1/exams`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/exams`	`CreateExamRequest`	`201 ExamResponse`	CENTER_ADMIN / SUPER_ADMIN
GET	`/api/v1/exams/{examId}`	—	`200 ExamResponse`	Any
GET	`/api/v1/exams?batchId={uuid}`	—	`200 List<ExamResponse>`	Any
PUT	`/api/v1/exams/{examId}/publish`	—	`200 ExamResponse`	CENTER_ADMIN / SUPER_ADMIN

QuestionController — `/api/v1/exams/{examId}/questions`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/exams/{examId}/questions`	`AddQuestionRequest`	`201 QuestionResponse`	CENTER_ADMIN / SUPER_ADMIN
GET	`/api/v1/exams/{examId}/questions`	—	`200 List<QuestionResponse>`	Any

EnrollmentController — `/api/v1/exams/{examId}/enrollments`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/exams/{examId}/enrollments`	`EnrollStudentRequest`	`201` `EnrollmentResponse`	STUDENT
DELETE	`/api/v1/exams/{examId}/enrollments/{enrollmentId}`	—	`204`	STUDENT (own)

SubmissionController — `/api/v1/exams/{examId}/submissions`

METHOD	PATH	REQUEST	RESPONSE	AUTH
POST	`/api/v1/exams/{examId}/submissions`	—	`201` `SubmissionResponse`	STUDENT
POST	`/api/v1/exams/{examId}/submissions/{submissionId}/answers`	`SubmitAnswersRequest`	`200` `SubmissionResponse`	STUDENT (own)
GET	`/api/v1/exams/{examId}/submissions/{submissionId}`	—	`200` `SubmissionResponse`	STUDENT (own) / SUPER_ADMIN

GradeController — `/api/v1/grades`

METHOD	PATH	REQUEST	RESPONSE	AUTH
GET	`/api/v1/grades?studentId={uuid}`	—	`200 List<GradeResponse>`	STUDENT (own) / SUPER_ADMIN
GET	`/api/v1/grades/{gradeId}`	—	`200 GradeResponse`	STUDENT (own) / SUPER_ADMIN

Error Contract (RFC 7807)

All errors return `application/problem+json`:

```
{
  "type": "https://edutech.com/problems/exam-not-found",
  "title": "Exam Not Found",
  "status": 404,
  "detail": "Exam with id 550e8400-e29b-41d4-a716-446655440000 not found"
}
```

EXCEPTION	HTTP STATUS	PROBLEM TYPE
`ExamNotFoundException`	404	`exam-not-found`
`QuestionNotFoundException`	404	`question-not-found`
`EnrollmentNotFoundException`	404	`enrollment-not-found`
`SubmissionNotFoundException`	404	`submission-not-found`
`AssessAccessDeniedException`	403	`access-denied`
`ExamNotPublishedException`	409	`exam-not-published`
`DuplicateEnrollmentException`	409	`duplicate-enrollment`
`SubmissionAlreadySubmittedException`	409	`submission-already-submitted`
`MaxAttemptsExceededException`	422	`max-attempts-exceeded`
`MethodArgumentNotValidException`	400	`validation-error`

10. Database Schema

Schema: `assess_schema` (PostgreSQL 15+)

V1 — Init Schema

```
CREATE SCHEMA IF NOT EXISTS assess_schema;
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
-- CREATE EXTENSION IF NOT EXISTS vector;  -- Deferred: enable when pgvector installed
```

V2 — `exams` Table

```
CREATE TABLE assess_schema.exams (
    id               UUID         PRIMARY KEY DEFAULT gen_random_uuid(),
    batch_id         UUID         NOT NULL,
    center_id        UUID         NOT NULL,
    title            TEXT         NOT NULL,
    description      TEXT,
    mode             TEXT         NOT NULL DEFAULT 'STANDARD',
    duration_minutes INT          NOT NULL CHECK (duration_minutes > 0),
    max_attempts     INT          NOT NULL DEFAULT 1 CHECK (max_attempts > 0),
    start_at         TIMESTAMPTZ,
    end_at           TIMESTAMPTZ,
    total_marks      NUMERIC(8,2) NOT NULL CHECK (total_marks > 0),
    passing_marks    NUMERIC(8,2) NOT NULL CHECK (passing_marks >= 0),
    status           TEXT         NOT NULL DEFAULT 'DRAFT',
    version          BIGINT       NOT NULL DEFAULT 0,
    created_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    updated_at       TIMESTAMPTZ  NOT NULL DEFAULT now(),
    deleted_at       TIMESTAMPTZ,
    CONSTRAINT chk_exam_mode   CHECK (mode   IN ('STANDARD', 'CAT')),
    CONSTRAINT chk_exam_status CHECK (status IN ('DRAFT', 'PUBLISHED', 'CLOSED', 'CANCELLED')),
    CONSTRAINT chk_passing_lte_total CHECK (passing_marks <= total_marks)
);
-- Partial indexes (active records only)
CREATE INDEX idx_exams_batch_id   ON assess_schema.exams(batch_id)  WHERE deleted_at IS NULL;
CREATE INDEX idx_exams_center_id  ON assess_schema.exams(center_id) WHERE deleted_at IS NULL;
CREATE INDEX idx_exams_status     ON assess_schema.exams(status)    WHERE deleted_at IS NULL;
-- Auto-update trigger
CREATE TRIGGER trg_exams_updated_at BEFORE UPDATE ON assess_schema.exams
    FOR EACH ROW EXECUTE FUNCTION assess_schema.set_updated_at();
```

V3 — `questions` Table

- Columns: `id, exam_id, question_text(TEXT), options_json(TEXT), correct_answer(INT), explanation(TEXT), marks, difficulty, discrimination, guessing_param, embedding_json(TEXT), version, created_at, updated_at, deleted_at`
- Indexes: `idx_questions_exam_id WHERE deleted_at IS NULL`
- BRIN on `created_at` for append-heavy query patterns

V4 — `exam_enrollments` Table

- Columns: `id, exam_id, student_id, status TEXT, enrolled_at, version, created_at, updated_at`
- Unique: `(exam_id, student_id)` — one enrollment per student per exam
- Partial index: `WHERE status = 'ENROLLED'`

V5 — `submissions` Table

- Columns: `id, exam_id, student_id, enrollment_id, attempt_number, started_at, submitted_at, total_marks, scored_marks, percentage, status, version, created_at, updated_at`
- Unique: `(exam_id, student_id, attempt_number)`
- Indexes: on `student_id`, on `exam_id WHERE status = 'IN_PROGRESS'`

V6 — `submission_answers` Table (IMMUTABLE)

```
CREATE TABLE assess_schema.submission_answers (
    id              UUID             PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id   UUID             NOT NULL REFERENCES assess_schema.submissions(id),
    question_id     UUID             NOT NULL REFERENCES assess_schema.questions(id),
    selected_option INT              NOT NULL CHECK (selected_option >= 0),
    is_correct      BOOLEAN          NOT NULL,
    marks_awarded   DOUBLE PRECISION NOT NULL DEFAULT 0,
    answered_at     TIMESTAMPTZ      NOT NULL DEFAULT now(),
    created_at      TIMESTAMPTZ      NOT NULL DEFAULT now()
    -- NO updated_at: this table is immutable by design
);
CREATE UNIQUE INDEX uq_submission_answers ON assess_schema.submission_answers(submission_id, question_id);
CREATE INDEX idx_submission_answers_submission_id ON assess_schema.submission_answers(submission_id);
CREATE INDEX idx_submission_answers_created_brin ON assess_schema.submission_answers USING BRIN(created_at);
```

V7 — `grades` Table

```
CREATE TABLE assess_schema.grades (
    id            UUID             PRIMARY KEY DEFAULT gen_random_uuid(),
    submission_id UUID             NOT NULL UNIQUE REFERENCES assess_schema.submissions(id),
    student_id    UUID             NOT NULL,
    exam_id       UUID             NOT NULL REFERENCES assess_schema.exams(id),
    batch_id      UUID             NOT NULL,
    center_id     UUID             NOT NULL,
    percentage    DOUBLE PRECISION NOT NULL,
    letter_grade  TEXT             NOT NULL,
    passed        BOOLEAN          NOT NULL,
    version       BIGINT           NOT NULL DEFAULT 0,
    created_at    TIMESTAMPTZ      NOT NULL DEFAULT now(),
    updated_at    TIMESTAMPTZ      NOT NULL DEFAULT now(),
    CONSTRAINT chk_letter_grade CHECK (letter_grade IN ('A', 'B', 'C', 'D', 'F'))
);
CREATE INDEX idx_grades_exam_id    ON assess_schema.grades(exam_id);
CREATE INDEX idx_grades_student_id ON assess_schema.grades(student_id);
CREATE INDEX idx_grades_batch_id   ON assess_schema.grades(batch_id);
CREATE TRIGGER trg_grades_updated_at BEFORE UPDATE ON assess_schema.grades
    FOR EACH ROW EXECUTE FUNCTION assess_schema.set_updated_at();
```

Index Strategy Summary

TABLE	INDEX TYPE	PURPOSE
exams	B-tree partial (`deleted_at IS NULL`)	Active exam lookups
questions	B-tree partial (`deleted_at IS NULL`)	Per-exam question lists
exam_enrollments	Unique B-tree `(exam_id, student_id)`	Duplicate enrollment prevention
submissions	Unique B-tree `(exam_id, student_id, attempt_number)`	Attempt uniqueness
submission_answers	Unique B-tree `(submission_id, question_id)`	One answer per question

TABLE	INDEX TYPE	PURPOSE
submission_answers	BRIN `created_at`	Append-only access pattern
grades	B-tree `student_id`, `exam_id`, `batch_id`	Grade reporting queries

11. Infrastructure Layer

11.1 JWT Security (RS256)

```
// JwtTokenValidator — JJWT 0.12.x
Jwts.parser()
    .verifyWith(publicKey)          // RSA public key from PEM file
    .requireIssuer(issuer)
    .build()
    .parseSignedClaims(token)
    .getPayload()                   // Claims → AuthPrincipal record
```

Claims mapping:
- `sub` → `userId` (UUID)
- `email` → `email`
- `role` → `role` (enum: STUDENT | CENTER_ADMIN | SUPER_ADMIN)
- `centerId` → `centerId` (nullable UUID)
- `deviceFP` → `deviceFP`

JwtAuthenticationFilter extracts `Authorization: Bearer <token>`, validates, sets `SecurityContextHolder`, adds `userId` and `role` to MDC for structured logging.

11.2 SecurityConfig

```
@EnableWebSecurity
@EnableMethodSecurity
public class SecurityConfig {
    // All endpoints require authentication
    // Stateless session management
    // CSRF disabled (JWT-based API)
    // JwtAuthenticationFilter added before UsernamePasswordAuthenticationFilter
}
```

11.3 Persistence Adapters — Pattern

Each persistence adapter:
1. Implements the domain `*Repository` interface
2. Is annotated `@Component`
3. Delegates to a **package-private** `SpringData*Repository extends JpaRepository`
4. Applies soft-delete filter: `WHERE deleted_at IS NULL` in all JPQL/derived queries
5. Never exposes Spring Data interfaces outside `infrastructure.persistence` package

```
// Example pattern
@Component
public class ExamPersistenceAdapter implements ExamRepository {
    private final SpringDataExamRepository springData;  // package-private

    public Optional<Exam> findById(UUID id) {
        return springData.findByIdAndDeletedAtIsNull(id);
```

```
    }
}
```

11.4 Kafka Configuration

Producer:

```
key-serializer: StringSerializer
value-serializer: JsonSerializer
properties:
  spring.json.add.type.headers: false   # clean JSON, no Spring type metadata
```

Consumer (CenterEventConsumer):

```
@KafkaListener(topics = "${kafka.topics.center-events}")
// StringDeserializer override for this consumer to avoid type header conflicts
// Listens for center/batch events to update local cache or trigger cross-service workflows
```

AssessEventKafkaAdapter:

```
@Component
public class AssessEventKafkaAdapter implements AssessEventPublisher {
    // KafkaTemplate<String, Object>
    // Routes by event type to configured topic
    // Catches and logs exceptions — NEVER re-throws (best-effort semantics)
}
```

Topic properties:

```
kafka:
  topics:
    assess-events: ${KAFKA_TOPIC_ASSESS_EVENTS}
    center-events: ${KAFKA_TOPIC_CENTER_EVENTS}
    audit-immutable: ${KAFKA_TOPIC_AUDIT_IMMUTABLE}
```

11.5 WebSocket / STOMP (Phase 1 Stub)

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {
    // SockJS endpoint: /ws/exams
    // Simple in-memory broker (no external relay in Phase 1)
    // Application destination prefix: configurable via ${ASSESS_SVC_WS_DESTINATION_PREFIX}
    // Phase 2: Replace with full STOMP relay (ActiveMQ / RabbitMQ)
}
```

---

12. Configuration Reference — All Environment Variables

Core Service

VARIABLE	USED IN	DESCRIPTION
`ASSESS_SVC_NAME`	`spring.application.name`	Service name for metrics/tracing
`ASSESS_SVC_PORT`	`server.port`	HTTP listen port

Database

VARIABLE	DESCRIPTION
`POSTGRES_HOST`	PostgreSQL hostname
`POSTGRES_PORT`	PostgreSQL port (default 5432)
`ASSESS_SVC_DB_NAME`	Database name
`ASSESS_SVC_DB_USERNAME`	DB user
`ASSESS_SVC_DB_PASSWORD`	DB password
`ASSESS_SVC_DB_POOL_MAX_SIZE`	HikariCP max pool size
`ASSESS_SVC_DB_POOL_MIN_IDLE`	HikariCP minimum idle connections
`ASSESS_SVC_DB_CONNECTION_TIMEOUT_MS`	HikariCP connection timeout
`ASSESS_SVC_DB_IDLE_TIMEOUT_MS`	HikariCP idle timeout

Redis

VARIABLE	DESCRIPTION
`REDIS_HOST`	Redis hostname
`REDIS_PORT`	Redis port
`REDIS_PASSWORD`	Redis password
`REDIS_SSL_ENABLED`	TLS toggle

Kafka

VARIABLE	DESCRIPTION
`KAFKA_BOOTSTRAP_SERVERS`	Kafka broker addresses
`ASSESS_SVC_KAFKA_CONSUMER_GROUP`	Consumer group ID
`KAFKA_TOPIC_ASSESS_EVENTS`	assess-svc outbound topic
`KAFKA_TOPIC_CENTER_EVENTS`	center-svc inbound topic
`KAFKA_TOPIC_AUDIT_IMMUTABLE`	Audit log topic

WebSocket / STOMP

VARIABLE	DESCRIPTION
`ASSESS_SVC_WS_DESTINATION_PREFIX`	STOMP application destination prefix
`ASSESS_SVC_WS_RELAY_HOST`	Message broker relay host (Phase 2)
`ASSESS_SVC_WS_RELAY_PORT`	Message broker relay port (Phase 2)

JWT

VARIABLE	DESCRIPTION
`JWT_PUBLIC_KEY_PATH`	Path to RSA public key PEM file
`JWT_ISSUER`	Expected issuer claim in JWT

AI / pgvector

VARIABLE	DESCRIPTION
`AI_EMBEDDING_DIMENSIONS`	Embedding vector dimensions (e.g. 1536 for OpenAI)
`AI_GATEWAY_BASE_URL`	Base URL of ai-gateway-svc

CAT (Computer Adaptive Testing)

VARIABLE	DESCRIPTION
`CAT_MIN_QUESTIONS`	Minimum questions before convergence check
`CAT_MAX_QUESTIONS`	Maximum questions per CAT session
`CAT_INITIAL_THETA`	Initial student ability estimate (typically 0.0)
`CAT_CONVERGENCE_THRESHOLD`	Theta change threshold to stop adapting

Resilience4j

VARIABLE	DESCRIPTION
`R4J_CB_AI_WINDOW_SIZE`	Circuit breaker sliding window size
`R4J_CB_AI_FAILURE_THRESHOLD`	Failure rate threshold (%)
`R4J_CB_AI_WAIT_DURATION`	Open-state wait before half-open

Observability

VARIABLE	DESCRIPTION
`ACTUATOR_ENDPOINTS`	Exposed actuator endpoints
`APP_ENVIRONMENT`	Environment tag for metrics
`OTEL_SAMPLING_PROBABILITY`	OpenTelemetry trace sampling rate
`OTEL_EXPORTER_OTLP_ENDPOINT`	OTLP collector endpoint
`LOG_LEVEL_ROOT`	Root logger level
`LOG_LEVEL_APP`	`com.edutech` logger level

---

13. Test Coverage

Test Run Results — 2026-03-07 — 11/11 PASS — BUILD SUCCESS — 6.412 s

ArchitectureRulesTest — 5/5 PASS

```
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.622 s
-- in com.edutech.assess.architecture.ArchitectureRulesTest
```

RULE	RESULT
domain must not depend on infrastructure or api	PASS
application must not depend on infrastructure or api	PASS
infrastructure must not depend on api	PASS
api must not depend on infrastructure	PASS
services must reside in application.service	PASS

SubmissionServiceTest — 6/6 PASS

```
[INFO] Tests run: 6, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 1.248 s
-- in com.edutech.assess.application.service.SubmissionServiceTest
```

TEST	SCENARIO	VERIFIED
`startSubmission_success`	PUBLISHED exam, enrolled student, 0 existing → IN_PROGRESS, attemptNumber=1	Response status, attemptNumber
`startSubmission_notEnrolled`	Student not enrolled → `EnrollmentNotFoundException`	Exception type, never saves
`startSubmission_maxAttemptsExceeded`	Attempts exhausted (count ≥ maxAttempts=1) → `MaxAttemptsExceededException`	Exception type
`submitAnswers_success`	2 correct answers (5+5 marks) → GRADED, scoredMarks=10.0, 2 events	Status, scoredMarks, `verify(eventPublisher, times(2))`
`submitAnswers_alreadySubmitted`	Submission already GRADED → `SubmissionAlreadySubmittedException`	Exception type, answerRepository never called
`submitAnswers_notOwner`	Different userId → `AssessAccessDeniedException`	Exception type, answerRepository never called

Test infrastructure fix applied: `@MockitoSettings(strictness = Strictness.LENIENT)` added at class level to allow shared `mockPublishedExam()` / `mockEnrollment()` helpers across tests that each use a subset of the stubbed methods. All `when()` calls extracted to local variables before being passed to `thenReturn()` (pattern established in parent-svc: nested `when()` inside `Optional.of(mockHelper())` inside `thenReturn()` triggers Mockito UnfinishedStubbing).

---

14. Kafka Event Flow

```
assess-svc Producer (assess-events topic):
┌──────────────────────────────────────────┐
│ ExamPublishedEvent    → exam lifecycle   │
│ ExamSubmittedEvent    → after grading    │
│ GradeIssuedEvent      → after grading    │
└──────────────────────────────────────────┘

assess-svc Consumer (center-events topic):
┌──────────────────────────────────────────┐
│ CenterEventConsumer                      │
│ @KafkaListener(center-events)            │
```

```
│ Handles: batch created/updated events        │
│ Purpose: maintain local reference data       │
└──────────────────────────────────────────────┘
```

15. Failure Modes and Resilience

FAILURE	BEHAVIOR
DB down	HTTP 503 (HikariCP timeout) — Spring propagates `DataAccessException` → GlobalExceptionHandler returns 503
Redis down	Startup may fail if Spring Data Redis auto-configures eagerly — configure `spring.cache.redis.enable-statistics=false` if needed
Kafka publish failure	Logged at WARN level, DB transaction already committed, event silently dropped
Concurrent submission grading	`@Version` on Submission prevents two concurrent grade operations — second will receive `OptimisticLockingFailureException` → 409
Double enrollment attempt	`UNIQUE(exam_id, student_id)` constraint → `DuplicateEnrollmentException` → 409
Question not found during grading	`QuestionNotFoundException` → 404, entire submitAnswers transaction rolled back
pgvector not installed	`embedding_json` stored as TEXT, vector similarity search not available, no startup failure
AI gateway down	Circuit breaker opens after threshold, fast-fail with 503 for AI-dependent operations
WebSocket relay down	STOMP simple in-memory broker used as fallback (Phase 1 config)

16. Cross-Service Dependencies

```
assess-svc RECEIVES (by UUID reference):
  ← center-svc:  centerId, batchId
  ← user-svc:    studentId (via JWT sub claim)

assess-svc PUBLISHES TO (Kafka):
  → ai-gateway-svc:  GradeIssuedEvent (for AI learning recommendations)
  → audit pipeline:  ExamSubmittedEvent, GradeIssuedEvent (audit-immutable topic)
  → (future) notify-svc: GradeIssuedEvent (grade notification to parent/student)
```

assess-svc does NOT call any other service via HTTP. All cross-service communication is event-driven. The service is fully autonomous given only the JWT token (which carries centerId, role, userId).

17. CAT Engine — Architecture Decision

`CatProperties` (`minQuestions`, `maxQuestions`, `initialTheta`, `convergenceThreshold`) is registered as a `@ConfigurationProperties` bean in `application.config` (not `infrastructure.config`) because it is consumed by the application service layer (future `CatExamService`). Placing it in infrastructure would create an illegal dependency: application → infrastructure.

The IRT (Item Response Theory) parameters (`difficulty`, `discrimination`, `guessingParam`) are stored on the `Question` entity to enable the 3-Parameter Logistic (3PL) model:

```
P(correct | θ) = c + (1 - c) × [e^(a(θ-b)) / (1 + e^(a(θ-b)))]
```

Where: `θ` = student ability, `b` = difficulty, `a` = discrimination, `c` = guessing parameter.

The CAT algorithm (adaptive item selection, ability estimation) is deferred to Phase 2 but all data structures are in place.

---

18. pgvector — Architecture Decision

`embeddingJson` is stored as `TEXT` (serialized `float[]` as JSON) rather than using the native `vector` type. This is a deliberate Phase 1 decision:

Reasons:

1. `pgvector` Hibernate custom type requires `UserType` implementation — adds complexity
2. The `vector` PostgreSQL extension may not be available in all deployment targets
3. The TEXT placeholder allows AI-generated embeddings to be stored immediately
4. Migration to native `vector` type requires only: enable extension + `ALTER COLUMN` + re-create HNSW index

Upgrade path:

```
-- Phase 2 upgrade (V8 migration):
CREATE EXTENSION IF NOT EXISTS vector;
ALTER TABLE assess_schema.questions ADD COLUMN embedding vector(1536);
UPDATE assess_schema.questions SET embedding = embedding_json::vector WHERE embedding_json IS NOT NULL;
ALTER TABLE assess_schema.questions DROP COLUMN embedding_json;
CREATE INDEX ON assess_schema.questions USING hnsw(embedding vector_cosine_ops);
```

---

19. Dependency Inventory

Runtime Dependencies (key)

DEPENDENCY	VERSION	PURPOSE
spring-boot-starter-web	3.x	REST API
spring-boot-starter-data-jpa	3.x	Persistence
spring-boot-starter-security	3.x	JWT filter chain
spring-boot-starter-validation	3.x	Bean Validation
spring-boot-starter-actuator	3.x	Health/metrics
spring-boot-starter-data-redis	3.x	Cache
spring-kafka	3.x	Event publishing/consuming
spring-boot-starter-websocket	3.x	STOMP WebSocket
jjwt-api + jjwt-impl + jjwt-jackson	0.12.x	JWT RS256 validation
postgresql	latest	JDBC driver
flyway-core	latest	Schema migrations
jackson-databind	2.x	JSON (incl. options serialization)
springdoc-openapi-starter-webmvc-ui	2.x	Swagger UI

Test Dependencies

DEPENDENCY	PURPOSE
junit-jupiter	Test runner
mockito-junit-jupiter	Mock framework
archunit-junit5	Architecture enforcement
assertj-core	Fluent assertions

20. Known Constraints and Upgrade Paths

#	CONSTRAINT	UPGRADE PATH
1	`embedding_json` stored as TEXT	V8 migration: enable pgvector, `ALTER COLUMN`, HNSW index
2	WebSocket/STOMP uses in-memory broker	Phase 2: configure STOMP relay (ActiveMQ/RabbitMQ) via `ASSESS_SVC_WS_RELAY_*` env vars
3	CAT algorithm not yet implemented	Phase 2: add `CatExamService` consuming `CatProperties`; IRT data already in DB
4	`submission_answers` has no soft-delete	By design (immutable). Administrative deletion requires direct DB operation with explicit audit
5	No rate limiting on submission start	Phase 2: add Redis-based rate limiter per studentId
6	Cross-service data (centerId, batchId) not validated via API call	By design: JWT-encoded centerId is trusted; batchId validated indirectly via exam ownership

21. Construction Record

AGENT	RESPONSIBILITY	FILES PRODUCED
Agent 1 — Domain + Application	6 entities, 6 enums, 3 events, 6 ports IN, 7 ports OUT, 2 config, 11 DTOs, 10 exceptions, 5 services	~56 files
Agent 2 — Infrastructure	KafkaTopicProperties, JwtTokenValidator, JwtAuthenticationFilter, SecurityConfig, WebSocketConfig, 6 Spring Data repos, 6 adapter classes, AssessEventKafkaAdapter, CenterEventConsumer	~19 files
Agent 3 — API + Bootstrap + Migrations + Tests	AssessSvcApplication, GlobalExceptionHandler, 5 controllers, 7 SQL migrations, ArchitectureRulesTest, SubmissionServiceTest, application.yml	~17 files
Total		**92 files**

Post-agent fixes applied:

1. `SubmissionServiceTest` — extracted mock helper calls to local variables before passing to `thenReturn()` (nested Mockito stubbing pattern)
2. `SubmissionServiceTest` — added `@MockitoSettings(strictness = Strictness.LENIENT)` to allow shared multi-stub helpers across tests with different stub usage profiles
3. `submitAnswers_success` — replaced `mockPublishedExam()` with targeted inline mock (only `getTotalMarks`, `getPassingMarks`, `getBatchId`, `getCenterId`)

This document is complete and permanently frozen as of 2026-03-07.
assess-svc: 11/11 tests PASS — BUILD SUCCESS — 6.412 s